PURPOSE AND SCOPE

Build the execution system first.

A reviewed implementation plan for real jobs on Docker workers

Build a small Slurm-free system that accepts jobs, assigns them to worker nodes, launches real processes, and records their outcomes. Keep job selection behind a replaceable interface so FIFO and a future RL picker use the same execution path.

First execution demonstration (W4): one controller and two Docker workers execute a mixed workload, respect allocation capacity and retain logs and outcomes. **Accepted baseline (W6):** recovery checks plus repeatable replay and complete reports.

What this plan covers. Job and worker contracts, durable controller state, resource accounting, placement, process execution, networking, cancellation, reconciliation, Docker packaging, workload replay, and evaluation reports.

Working assumptions. One active controller; trusted Linux jobs; one worker per job; CPU and memory scheduling; several concurrent jobs per worker when requests fit. Python, SQLite, HTTP/JSON, and Docker Compose are proposed implementation defaults, not previously finalized language commitments.

Scope boundary. Training a new RL policy is a later workstream. This plan prepares its observation/action interface. Multi-machine jobs, GPU execution, hostile multi-user isolation, high-availability controllers, and a general-purpose cluster platform are later extensions. Work is sequenced by acceptance gates; no calendar or staffing commitment is assumed.

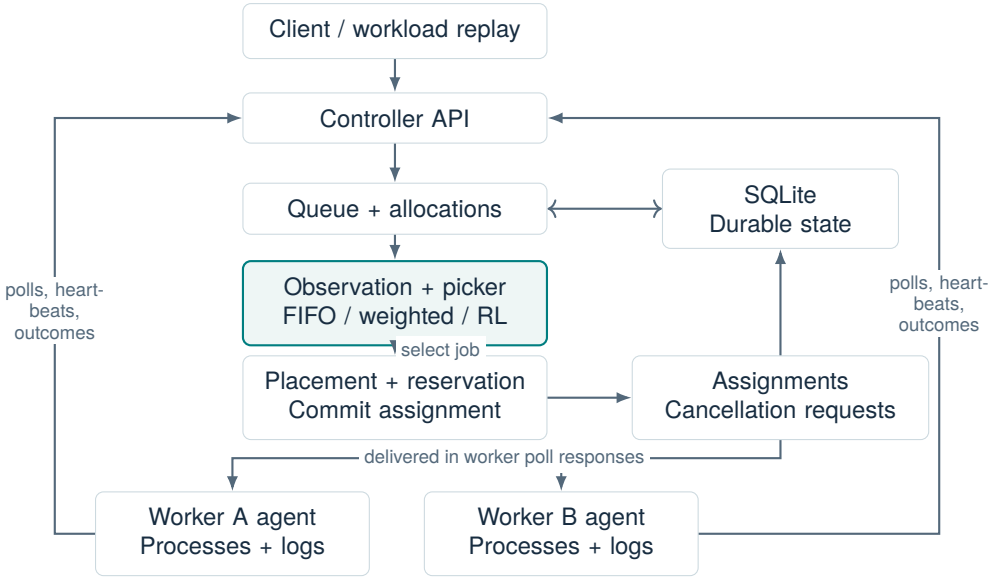
Current asset	Verified repository state	Use in the new system
Simulator	Event-based replay, resource constraints, fixed-weight scheduling and metrics.	Reuse concepts and tested resource checks; later expose decision boundaries.
Workload generator	Repeatable traces; includes runtime labels and may generate multi-node jobs.	Add single-worker execution manifests and real command workloads.
RL code	Gaussian weight-bandit training, one action per trace.	Preserve as an offline baseline; it is not the sequential picker.
Docker environment	Existing image and Compose services launch Slurm.	Add a separate custom-runner image and Compose configuration.
Execution system	No custom controller or worker agents yet.	New work specified by this plan.

Baseline and status. Repository HEAD is `06a6416`. Existing simulator edits and other local files remain uncommitted and must be preserved. PyTorch is absent from the current virtual environment; no model checkpoints were found. Those facts do not block building the execution path. Plan review findings and their disposition appear on page 9; they are design findings, not completed system tests.

ARCHITECTURE

Keep decisions separate from execution.

The controller owns scheduling state and assignments. Workers own local processes. The picker sees a snapshot and returns a choice; it does not mutate resource counters or send launch requests.



Central scheduling, worker pull. Workers register and poll with their session identity, active attempts, and unacknowledged events. The controller returns assignments already committed to that worker. A worker never independently removes the next job from the shared queue. Polling order must not become an accidental scheduling policy.

One decision boundary. On an arrival, outcome, worker-state change, or timer, build a snapshot containing all queued jobs and feasibility information. The picker returns `SelectJob(id)` or `Wait`. Revalidate the snapshot, choose a compatible worker deterministically, and commit the reservation before delivery. Waiting schedules a positive-time wakeup, including when no processes are running.

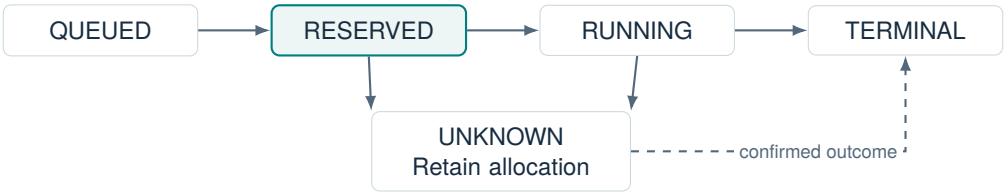
Docker's role. Compose supplies node containers and networking. Our controller assigns jobs. Each worker launches child processes inside its container; nested Docker and access to the host Docker socket are unnecessary for this prototype. Container limits constrain aggregate worker usage. Declared per-job allocations alone do not enforce individual process CPU or memory limits.

CONTRACTS AND STATE

Make uncertainty part of the model.

Separate the submitted job from an execution attempt. A new attempt is an explicit retry; repeated delivery of the same attempt is not permission to launch another process.

Contract	Minimum contents
JobSpec	Job ID and submission key; argument list; working directory; environment; CPU/memory request; execution timeout; optional duration estimate and input reference. Actual runtime is an outcome.
WorkerSession	Stable worker ID, current reporting session, execution-session identity, capacity, health and active attempt inventory. New registration does not erase unresolved commitments.
Attempt	Attempt ID, job ID, worker/session, state, reserved resources, cancellation request, start/finish evidence, exit status and output references.
Event	Stable event ID, attempt/session identity, event type, timestamps and payload. Worker retains events until acknowledged.
Observation	Snapshot version; all queued requests and ages; worker availability after commitments; in-flight/running attempts; feasible choices; timer/control-rule state. No future arrivals or true remaining runtimes.



Terminal outcomes: success, failure, timeout, cancellation, or LOST after confirmed environment destruction without an exit result. UNKNOWN may reconcile to RUNNING.

Four invariants govern every transition.

1. Reserve resources and record an attempt in one controller transaction before delivery. Capacity includes both reserved and running allocations.
2. Keep one unresolved attempt per job. Missing heartbeats make a worker unavailable; they do not prove that its processes stopped.
3. Release capacity once, after the managed process group is stopped or execution-environment loss is confirmed. Cancellation remains requested until termination is confirmed.
4. Deduplicate by identity. A current reporting session may reconcile older execution attempts, but cannot let stale messages overwrite newer attempts. Revalidate stale decisions.

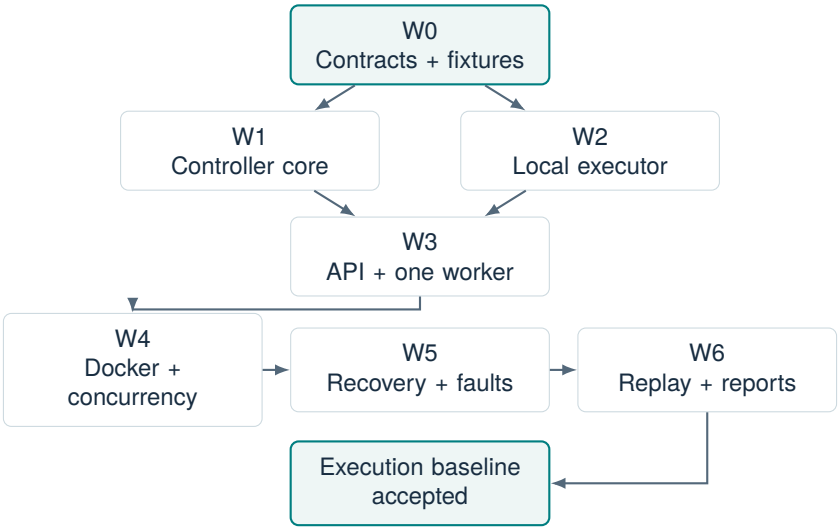
The launch gap needs a conservative rule. Persisting a worker launch record and creating an OS process are not one atomic operation. After a crash in that gap, a worker must reconcile process identity or report uncertainty; it must not blindly launch again. This plan promises explicit reconciliation, not blanket exactly-once execution.

Timing. Use monotonic time for local deadlines and elapsed measurements; persist UTC timestamps and event ordering for reports. On physical hosts, cross-host timestamps need clock synchronization or stated uncertainty. Docker's shared host is the first test environment, not evidence that distributed clock issues are solved.

DELIVERY ORDER

Use gates, then expand the system.

Contracts come first. Controller-state work and local-executor work can then proceed independently. Integrate a single worker before adding concurrent workers. Recovery semantics are specified early and exercised throughout; the recovery phase expands fault testing rather than adding correctness as an afterthought.



	Depends on	Primary owner role	Gate evidence
W0	Repo review	Interfaces / integration	Versioned contracts, lifecycle table and fixed fixtures.
W1	W0	Controller / storage	Transactional reservations and event reducer tests.
W2	W0	Worker / processes	Real process execution, logs, timeout and cancellation checks.
W3	W1, W2	Integration	Submit, execute, inspect and cancel through the API.
W4	W3	Runtime integration	Two Docker workers and concurrent placements without over-allocation.
W5	W4	Reliability / integration	Restart and message-loss scenarios with reconcilable outcomes.
W6	W5	Experiments / reporting	Repeatable workload replay and complete comparison reports.

Parallel work. W1 and W2 share contracts, not mutable state. The replay manifest and report schema can be designed during W3, while acceptance of experimental results waits for W5. Roles describe ownership boundaries and can be combined by one developer.

Planning discipline. Split each work package into reviewable changes with its gate tests. Re-estimate remaining work after the first end-to-end W3 run. Early measurements should determine polling intervals, launch overhead and test workload duration; this document does not invent throughput targets or delivery dates.

FOUNDATION WORK

Implement contracts, state and processes.

W0 Freeze the execution contracts

Deliver: shared types, API/event versioning, lifecycle transitions, units (CPU slots, MiB, seconds), validation rules and a small fixture workload. Define submission idempotency, worker/session identity and attempt IDs before transport code.

Specify input availability and outputs: an immutable shared input mount for the local prototype, per-attempt output directories, and bounded log retention or size limits. Document that trusted jobs are expected to honor declared per-job resource requests.

Gate: fixtures cover a valid job, a request exceeding every configured worker's total capacity (reject), a duplicate submission, a temporarily blocked queued job and a job with no runtime estimate. State diagrams explain cancellation before and after assignment. A configured worker inventory allows validation even when a worker is temporarily offline.

W1 Build the controller core and persistent store

Deliver: a single-writer controller state machine, SQLite schema/migrations, submission and event reducers, worker inventory, deterministic placement, and an assignment-delivery outbox. Use explicit transactions for job state, attempt creation and resource commitment. Rebuild authoritative state from persisted records on restart.

Expose a read-only scheduling snapshot and a picker interface. Preserve all queued jobs in the snapshot: strict FIFO must be able to wait behind a temporarily blocked oldest job. Store the policy version and selected action with each scheduling decision.

Gate: two competing assignments cannot exceed capacity; duplicate outcomes do not free resources twice; stale session events cannot mutate a newer attempt; repeated submissions resolve to the same job; restarting the controller preserves unresolved work.

W2 Build a local executor library

Deliver: process launch from an explicit argument list, configured environment and working directory; per-attempt stdout/stderr files; exit status; monotonic timeout; process-group cancellation with a grace period and forced termination when needed. Drain output continuously or write directly to files so verbose jobs cannot deadlock on pipes.

Retain an attempt journal and unacknowledged events. Associate process evidence with the attempt and execution session; a PID alone is insufficient after restart. Serialize launch and cancellation per attempt. Persist cancellation markers and check them before spawn so a delayed assignment cannot launch after cancellation is acknowledged.

Gate: real helper processes verify success, nonzero exit, launch failure, large output, timeout, cancel-before-delivery, cancel-during-launch and parent exit with a surviving child. Trusted jobs must not detach descendants from the managed process group. Terminate remaining managed descendants and confirm the group stopped before any terminal outcome releases capacity. Preserve the main exit result and record cleanup separately.

Integration rule: controller and executor changes can proceed in parallel after W0. Keep their tests independent: the controller uses a fake worker transport; the executor uses real local processes without requiring a controller or Docker.

RUNTIME WORK

Connect, package and recover.

W3 Add the API, worker agent and client

Deliver: submit/status/cancel/log-access operations; worker registration and polling; heartbeat/inventory exchange; assignment delivery; idempotent outcome ingestion. The agent wraps W2. Workers retain outcome events until the controller commits and acknowledges them. The client exposes job state and useful failure reasons.

Use an explicit command/event version and reject incompatible messages. Do not hold database transactions open while waiting for network delivery. A timed-out delivery keeps its existing attempt ID and reservation; it does not create a fresh attempt.

Gate: one controller and one worker run in separate processes. A client submits a job, retrieves output and outcome, cancels a second job, and reconnects after a dropped connection without duplicate launch. Reserved capacity is visible before RUNNING.

W4 Add Docker workers and concurrent execution

Deliver: a new custom-runner Dockerfile and Compose configuration with one controller and two workers, persistent controller/worker state, inputs and output mounts, health checks, and distinct worker identities. Existing Slurm files remain usable.

Configure worker-level CPU/memory limits and advertised schedulable capacities that leave memory/CPU headroom for the worker agent. Jobs run as child processes; Docker handles container limits, while the controller accounts for per-job requests. Use a single serialized decision/commit path even when multiple workers poll or report completions concurrently.

Gate: demonstrate execution on both workers, concurrent fitting jobs on one worker, queueing when capacity is occupied, cancellation with later capacity reuse, and a resource request that can fit only one configured worker. Confirm host capacity is sufficient for the test; virtual quotas are not dedicated physical cores.

W5 Verify recovery and lifecycle corner cases

Deliver: reconciliation for controller restart, stale worker sessions, disconnects, lost launch acknowledgments and duplicate reports. A disconnected worker is unavailable for new assignments. New registration retains prior unresolved commitments. The current reporting session may reconcile old execution-session attempts without modifying newer attempts. Retry is explicit after the previous attempt is confirmed terminal.

Test crash points before/after reservation commit, delivery, launch journaling, process creation and result acknowledgment. Reconciliation may require operator resolution in the prototype when process evidence is insufficient. Do not turn uncertainty into an unjustified success, failure, or second launch.

Gate: the fault matrix on the next page passes for supported cases. A controller restart rebuilds the same unresolved allocations; terminal events survive worker/API reconnection; cancellation during launch cannot be reported complete while work remains alive.

VERIFICATION AND EXPERIMENTS

Make every claim observable.

Tests should check externally meaningful outcomes and state invariants. Existing simulator tests remain a regression gate when shared helpers are changed; no production code should be rewritten merely to share a name or schema.

Scenario	Required assertion	Evidence
Concurrent dispatch	Commitments never exceed configured capacity.	Job/attempt state plus reservation ledger.
Repeated assignment	At most one actual launch per attempt in an uninterrupted session; an ambiguous crash gap triggers no new spawn.	Job-written start markers, process evidence and explicit UNKNOWN when needed.
Duplicate/out-of-order events	No double release or terminal-state regression.	Event IDs, transition history and final ledger.
Cancel / timeout	Confirm managed group termination; a delayed assignment cannot launch after cancellation.	Durable cancellation marker, child-process checks and timestamps.
Controller restart	Unresolved work and commitments survive.	Before/after inventory and persistent records.
Worker disconnect	No new assignments to that session; no automatic duplicate retry.	Health state and unchanged unresolved attempt ID.
No running jobs + Wait	Another decision is scheduled; no busy loop or deadlock.	Fake-clock event test and live timer test.
Strict FIFO head blocked	Later jobs do not bypass the head.	Ordered decisions and identical placement path.

W6 Build replay and complete reports

Deliver: an execution manifest containing arrivals, command templates, resources, limits, seeds and input references. Generate single-worker jobs explicitly. Use sleep jobs for lifecycle checks and CPU/memory helpers for execution experiments; a synthetic runtime label is not itself an executable workload or a guaranteed wall-clock duration.

Capture controller-received submission time, reservation, launch/start evidence, finish, exit status, worker/session, attempt ID, policy/configuration versions and resource allocations. Report submission-to-completion response time, queue/start delay, throughput, tail waiting and allocation utilization. Label allocation utilization separately from measured CPU/memory usage.

Comparison protocol: reuse the same controller, worker code, placement, failure rules and measurements; change only the picker. Replay identical manifests from equivalent clean starting conditions. Alternate policy run order and repeat runs to expose shared-host variation. Record measured submission timing as well as intended arrivals.

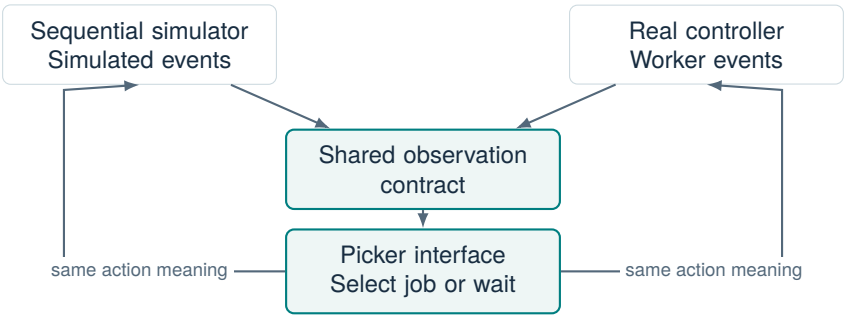
Gate: reports account for every submitted job, including failures and unfinished work. Complete successful workloads support response-time comparisons. Incomplete or failed runs remain clearly marked; averaging only successful jobs must not be presented as an improvement. Check that admission delay and launch latency remain in response time.

Baseline naming: strict FIFO waits if the oldest queued job cannot fit. Selecting the oldest job that fits is a separate baseline. Both can use the same system, but their behavior and names must not be conflated.

FUTURE PICKER HANDOFF

Prepare the interface without building RL now.

The execution project ends with a dependable policy-independent runner. A subsequent workstream can implement a sequential simulator and RL picker against the same contract. The model controls job selection and waiting; placement, reservations, launch, cancellation and reconciliation remain execution-system responsibilities.



Required compatibility. Expose all queued requests with feasibility information, including committed-but-not-started allocations. Use stable job IDs and a snapshot version. A stale policy response is revalidated or recomputed before commitment. Do not expose actual runtime, exact remaining duration, or future arrivals to the policy.

Waiting and progress. Wait ends at the next arrival, completion, worker-state change, or positive control tick. Any operational limit on voluntary idling must be configured, observed by the policy, shared across simulation and execution, and logged. Never infer that waiting is invalid from hidden knowledge that a trace has no future jobs.

Simulation changes later. Existing `simulate()` runs a complete fixed-weight trace and rejects some idle states that a waiting policy needs. The new environment must expose decisions and account for launch delay and committed resources. Preserve the legacy replay path and its regression tests. Match logical semantics, not exact wall-clock timestamps.

Research distinction. DeepRM selects a job for earliest feasible future placement in a pooled resource calendar. Our proposed first picker selects a job for dispatch now on an individual worker. Training must reflect this adaptation. The current Gaussian weight-bandit network and full-trace features are not a drop-in sequential controller.

Metrics are ready for future rewards. For complete episodes, the integral of arrived, unfinished jobs measures total response time:

$$\int N_{\text{unfinished}}(t)\,dt = \sum_j (t_{\text{finish},j} - t_{\text{submit},j}).$$

Queued, reserved and running jobs all count. Dividing by the number of jobs gives mean response time. Record failures and incomplete work explicitly rather than allowing them to disappear from a future reward or comparison.

REVIEW AND REPOSITORY MAP

Close the design gaps before implementation.

Review status: two independent agent reviews assessed architecture/lifecycle and implementation/test design. Both found the overall architecture and delivery order sound. The corrections below are incorporated. Review assessed the plan only; no runtime or system tests were implemented as part of this document.

Issue examined	Disposition in this plan
Session reconciliation	Separate reporting from execution sessions; reconcile old attempts while preserving unresolved commitments and protecting newer attempts.
Delayed assignment after cancel	Persist cancellation markers before acknowledgment; serialize launch/cancel and check the marker before spawn.
Children outlive parent	Trusted jobs stay in the managed process group. Clean up remaining descendants before terminal reporting and capacity release.
Weak duplicate-launch test	Count actual starts using job-written markers and process evidence, rather than counting acknowledgments.
Acceptance allows uncertainty	Normal fixtures must reach their expected terminal outcomes with no UNKNOWN. Uncertainty is limited to explicit ambiguous fault tests.
Impossible versus blocked	Reject requests exceeding all configured workers; queue temporarily blocked jobs. Strict FIFO preserves head-of-queue blocking.
Milestone ambiguity	W4 is the first two-worker demonstration; W6 is the accepted baseline with recovery evidence and complete reports.

Proposed module boundaries. Names below describe new work, not existing files:

Proposed location	Responsibility
`execution/models.py`	Job, worker, attempt, event and observation contracts.
`execution/store.py`, `controller.py`	Durable transitions, reservations, decisions and reconciliation.
`execution/executor.py`, `worker.py`	Local process lifecycle, polling and event delivery.
`execution/api.py`, `cli.py`	Controller transport and user operations.
`execution/replay.py`, `reporting.py`	Workload manifests, replay and complete reports.
`compose.execution.yaml`	Separate controller/worker deployment for this system.

Reuse carefully. Resource compatibility and placement ideas live in `src/slurm_aware/simulator.py`; trace generation is in `workloads.py`. The existing `rl/` package is the weight-bandit baseline. Extract shared helpers only when tests demonstrate unchanged behavior. Existing local changes, legacy runtime-model experiments, Slurm Docker files and PDFs are not cleanup targets for this work.

First implementation batch. W0 contracts and fixtures, followed by independent W1 controller-state and W2 executor changes. Integrate only after their gates pass. The first demonstration uses the same system with a simple picker; RL is a later swap.

ACCEPTANCE AND REFERENCES

Define what a finished first system means.

Execution baseline acceptance. A fresh checkout plus documented dependencies can start the new Compose setup. Submit a fixed mixed workload, inspect jobs and workers, retrieve outputs, cancel a running job and observe correct capacity reuse. Run the same workload again from clean state and produce another complete report.

- The normal fixture workload reaches every expected terminal outcome (including intentional failures, timeouts and cancellations), with no unresolved attempts.
- UNKNOWN is an expected outcome only for explicitly injected ambiguous faults. Every job and attempt remains traceable to its outcome or recorded uncertainty.
- Resource commitments include delivery/startup, remain within configured capacity, and are released once after definitive evidence.
- Restart and delivery fault tests demonstrate supported recovery behavior without claiming that every arbitrary crash can be resolved automatically.
- Reports identify the workload, configuration, policy version, environment and all outcomes.
- FIFO and any future picker share the execution, placement, timeout and reporting paths.

Decisions to record in W0. Confirm Python-first implementation; select worker capacities and input/output mounts for the available host; choose heartbeat and decision timer defaults through measurement; specify the trusted-job resource contract and the procedure for unresolved attempts. These are configuration/design decisions, not reasons to begin with a larger distributed platform.

Evidence used. Repository inspection at HEAD `06a6416`, including the dirty working tree, informed the reuse boundaries. The existing simulator and documentation provide context; this plan does not claim that their tests validate the new runtime.

[1] **Current simulator:** `src/slurm_aware/simulator.py`. The live-runtime separation follows from `Job` requiring runtime and rejecting runtime above its limit; `simulate()` currently controls a complete replay.

[2] **Current workloads and learner:** `src/slurm_aware/workloads.py` and `src/slurm_aware/rl/`. Generator outputs include multi-node jobs; current training uses one Gaussian weight action per trace, with hindsight-derived features.

[3] **DeepRM:** H. Mao, M. Alizadeh, I. Menache and S. Kandula, *Resource Management with Deep Reinforcement Learning*, HotNets 2016. Local source: `deeprm_hotnets16.pdf`. Sections 3 and 5 inform the action semantics and the limitations of pooled resources and known durations.

[4] **Python subprocess execution:** Python 3.11 asyncio subprocess documentation. Supports the proposed asynchronous worker executor.

[5] **Docker networking:** Networking in Compose. Supports the local controller/worker service topology.

[6] **Docker limits:** Resource constraints. Supports worker-container CPU/memory limits; it does not make reservations into per-job limits.

Document reproduction. The accompanying `scheduler-work-plan.tex` contains the text and TikZ diagrams. Compile with:

```
tectonic --untrusted --keep-logs scheduler-work-plan.tex
```

This PDF is a reviewed work plan. Creating it did not implement the controller, launch workers, alter application code, install RL dependencies, or start containers.